

UNIVERSITÀ DEGLI STUDI DI SALERNO

Dipartimento di Informatica

Corso di Laurea Triennale in Informatica

TESI DI LAUREA

AmbraIA: Memoria, apprendimento e pianificazione per NPC in ambienti 3D con LLM locale

RELATORE

Prof. Fabio Palomba

CORRELATORI

Dott. Antonio Della Porta

Dott.ssa Viviana Pentangelo

CANDIDATO

Giovanni De Caro

Matricola: 0512119354

Anno Accademico 2026-2027

Questa tesi è stata realizzata nel



Abstract

La creazione di sistemi decisionali per personaggi non giocanti (NPC) nei videogiochi è stata tradizionalmente affrontata tramite macchine a stati e alberi decisionali, strumenti che generano comportamenti rigidi, faticano ad adattarsi a contesti non previsti e mancano di meccanismi di memoria episodica e apprendimento continuo. Questo lavoro di tesi presenta un'architettura per NPC in ambienti 3D che integra memoria persistente, retrieval semantico delle informazioni, pianificazione gerarchica e riflessione periodica sui propri comportamenti. Il sistema utilizza un modello linguistico in esecuzione locale, senza dipendenze da API cloud, e include un meccanismo di adattamento automatico a scene non preconfigurate. I risultati, ottenuti tramite simulazioni condotte in ambiente Unity, evidenziano che l'NPC sviluppa comportamenti coerenti con i propri bisogni fisiologici dinamici, manifesta capacità di apprendimento attraverso le riflessioni e si adatta ad ambienti diversi senza richiedere configurazioni manuali. L'architettura proposta risulta modulare, estensibile e completamente autonoma da servizi esterni.

Indice

Elenco delle Figure **iii**

Elenco delle Tabelle **iv**

1 Introduzione **1**

1.1 Non-Playable Character: contesto e ruolo 1

1.2 Obiettivi 2

1.3 Panoramica dei risultati 2

1.4 Struttura della tesi 3

2 Dagli NPC tradizionali agli LLM: background e stato dell'arte **4**

2.1 NPC tradizionali: analisi e limiti 4

2.1.1 Alberi decisionali 6

2.1.2 Macchine a stati finiti 7

2.1.3 Limiti degli approcci tradizionali 8

2.2 Gli LLM nei videogiochi: opportunità e sfide 8

2.2.1 LLM come strumenti per lo sviluppo di videogiochi 8

2.2.2 LLM per il controllo degli NPC 9

2.2.3 Sfide e limiti degli LLM 9

3 AmbraIA: dall'analisi del problema all'architettura **11**

3.1 Specifica PEAS 11
3.2 Caratteristiche dell'ambiente 12
3.3 Analisi del problema 13
3.4 Architettura del modulo Unity 14
3.4.1 NPCController 15
3.4.2 InteractableObject 16
3.4.3 SceneAnalyzer 16
3.4.4 RuntimeCompiler 17
3.5 Architettura del modulo server 17
3.5.1 Memory Stream 19
3.5.2 Retrieval 21
3.5.3 Planner 22
3.5.4 Builder 23
3.5.5 Inspector 25
3.5.6 Reflector 26
3.6 Configurazione scena 26

4 Sperimentazione e Risultati **28**
4.1 Setup sperimentale 28
4.2 Criteri di valutazione 30
4.3 Analisi dei risultati 31
4.3.1 Coerenza delle azioni 31
4.3.2 Varietà comportamentale 32
4.3.3 Tasso di successo della compilazione 32
4.3.4 Adattabilità a scene nuove 32
4.3.5 Latenza 33
4.3.6 Considerazioni generali 33

5 Conclusioni **35**
5.1 Considerazioni finali 35
5.2 Sviluppi futuri 35

Bibliografia **37**

Elenco delle figure

2.1 Struttura a moduli dell'IA nei videogiochi. Figura adattata da [1]. . . 5
2.2 Esempio di albero decisionale per un NPC. 6
2.3 Esempio di macchina a stati finiti per un NPC. 7

3.1 Panoramica del lato Unity di AmbraIA. 15
3.2 Panoramica del lato server di AmbraIA. 19

4.1 La scena di test con l'NPC. 30

Elenco delle tabelle

4.1 Configurazione del setup sperimentale. 29
4.2 Criteri di valutazione adottati. 31
4.3 Latenza media per nodo del grafo. 33

CAPITOLO 1

Introduzione

1.1 Non-Playable Character: contesto e ruolo

I personaggi non giocanti (NPC) sono elementi fondamentali nel mondo videoludico. Sono parte integrante di un mondo interattivo e contribuiscono alla credibilità del mondo stesso. Ogni videogioco costruisce l'architettura dei propri NPC in base all'obiettivo del gioco: se parliamo di un gioco prettamente d'azione dove gli NPC rappresentano minacce per il giocatore, come per esempio nella maggior parte dei giochi della serie *Super Mario*, non è necessario che essi si comportino in maniera "umana", replicando tratti e azioni tipiche delle persone, ma devono semplicemente seguire il giocatore per attaccarlo. In altri videogiochi invece, come *The Sims*[2] o *Tomodachi Life*[3], dove si simula la quotidianità della vita di un essere umano, gli NPC sono il fulcro del design dei giochi: essi devono essere in grado di agire tra di loro e nell'ambiente circostante, reagire a stimoli fisiologici interni e interagire in modo credibile con l'ambiente. In pratica, si cerca di simulare il comportamento umano. Tuttavia, la mente umana è molto complessa, e rappresentare i comportamenti che il pensiero genera è altrettanto complicato.

1.2 Obiettivi

AmbraIA si pone l'obiettivo di creare NPC capaci di agire in ambienti virtuali 3D differenti, principalmente su Unity. L'NPC reagisce agli elementi che compongono la scena, apprende le varie interazioni, soddisfa i propri bisogni fisiologici e agisce in base ai propri ricordi ed esperienze passate. In pratica, ciò che AmbraIA si propone è simulare comportamenti umani adattabili a contesti 3D diversi, seppur in forma semplificata rispetto alla complessità della mente umana. Un esempio concreto: se l'NPC tende a mangiare solamente dolci, esaminando i propri ricordi riuscirà a capire che deve variare la propria dieta. Questo comportamento è reso possibile dall'uso di modelli linguistici locali, che supportano il sistema nel prendere decisioni, generare codice dinamico e riflettere sulle azioni passate, rendendo superflue, in questo contesto, strutture decisionali rigide come le macchine a stati finiti. Inoltre, il progetto permette di adattare una scena al sistema decisionale in modo automatico tramite interfacce dedicate. Questo non preclude la possibilità di modificare la scena in un secondo momento, cambiando animazioni, interazioni o gli stessi *GameObject*. AmbraIA punta quindi fortemente sulla flessibilità: l'obiettivo non è funzionare solo con determinati tipi di scene o videogiochi, ma simulare in modo semplificato il comportamento umano su scene diverse, che possono essere utilizzate per creare il proprio videogioco o, più in generale, un mondo virtuale più immersivo.

1.3 Panoramica dei risultati

Valutare un NPC non è come valutare un algoritmo di classificazione: non esistono metriche standard come accuracy o precision, perché il comportamento di un agente virtuale non è univocamente giusto o sbagliato. La valutazione di AmbraIA si basa quindi principalmente su criteri qualitativi, osservando se il comportamento prodotto risulta coerente, vario e adattivo rispetto alla situazione. Fanno eccezione la latenza e il tempo di configurazione, che sono intrinsecamente quantitativi.

AmbraIA è stata testata principalmente su una scena 3D composta da sei oggetti interagibili. Il sistema ha mostrato una buona coerenza comportamentale con lo stato fisiologico e una discreta varietà comportamentale, grazie soprattutto al sistema di

estrazione dei ricordi (`Retrieval`) e al modulo che genera riflessioni periodiche (`Reflector`). L'adattabilità è stata valutata separatamente su tre scene diverse, testando il meccanismo di configurazione automatica: in media, ogni scena è stata configurata in circa 1440 ms. La criticità maggiore riguarda la latenza, che si attesta su una media di circa 57 secondi. Tutti questi aspetti vengono approfonditi nel Capitolo 4.

1.4 Struttura della tesi

Nei prossimi capitoli si entrerà nel dettaglio dell'IA per gli NPC nei videogiochi e della descrizione dell'architettura di AmbraIA. L'elaborato è strutturato come segue:

- **Capitolo 2 — Stato dell'arte:** analisi della documentazione scientifica e delle metodologie per l'IA degli NPC, partendo dalle strutture tradizionali fino all'integrazione dei Large Language Models.
- **Capitolo 3 — Analisi e Architettura:** studio del problema affrontato da AmbraIA e trattazione dettagliata della sua architettura e del suo funzionamento interno.
- **Capitolo 4 — Sperimentazione e Risultati:** approfondimento dei test effettuati e dei risultati ottenuti, valutando le prestazioni dell'agente.
- **Capitolo 5 — Conclusioni e Sviluppi Futuri:** considerazioni finali sul lavoro svolto, analisi delle limitazioni riscontrate e possibili direzioni per future integrazioni.

CAPITOLO 2

Dagli NPC tradizionali agli LLM: background e stato dell'arte

2.1 NPC tradizionali: analisi e limiti

L'IA nei videogiochi è nata praticamente in simbiosi con i videogiochi stessi. Basti pensare a *Pong*[4], in cui l'avversario deve cercare di colpire la pallina evitando di subire punto. Un altro esempio importante è *Pac-Man*[5] del 1980, dove i quattro fantasmi hanno ciascuno un algoritmo differente per raggiungere il protagonista: Blinky, il fantasma rosso, imposta il suo obiettivo di movimento in base alla posizione attuale di Pac-Man, mentre Inky, il fantasma blu, si basa sulla posizione del fantasma rosso e di Pac-Man per accerchiarlo. Inizialmente gli NPC servivano principalmente per dare una sfida al giocatore. Oggi troviamo sistemi più complessi, alcuni dei quali simulano anche il comportamento umano, come *The Sims*[2], che simula la vita quotidiana delle persone.

Come osserva Millington: "Now we have a massive diversity of AI in games. Many genres are still using the simple AI of 1979 because that's all they need" [1]. Nonostante l'IA nei videogiochi, e più in generale nel mondo dell'informatica, si sia evoluta molto, vari videogiochi moderni utilizzano ancora strutture vecchie di

decenni. Il motivo risiede nella diversità dei generi videoludici e nello scopo che ogni videogioco si prefigge, come già introdotto nel Capitolo 1. Normalmente un'IA per un NPC viene strutturata in tre moduli principali: Movimento, Decision Making e Strategia, come mostrato in Figura 2.1. Questa è una struttura proposta da Ian Millington per spiegare meglio il concetto di IA nei videogiochi [1]. I primi due moduli agiscono sul singolo agente, mentre la Strategia agisce anche sulle azioni degli altri agenti.

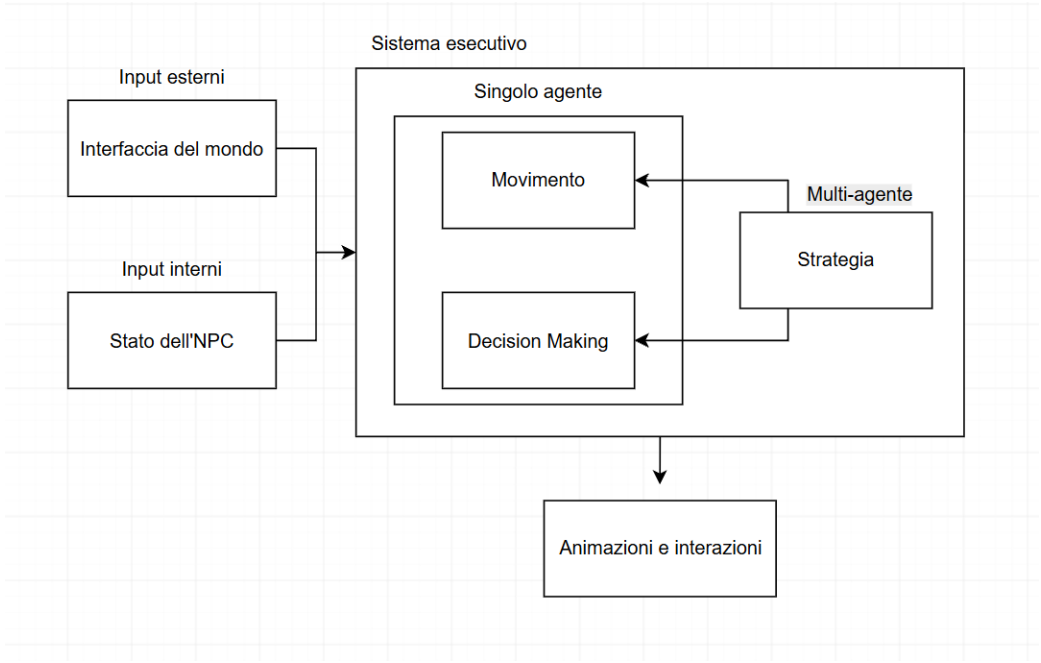


Figura 2.1: Struttura a moduli dell'IA nei videogiochi. Figura adattata da [1].

Il Movimento è definito come il modulo che calcola un certo tipo di percorso e coordina il movimento dell'NPC. Il Decision Making permette all'NPC di decidere quali azioni compiere tra quelle disponibili. La Strategia serve a coordinare i due moduli sopracitati per farli lavorare insieme ad altri agenti. Non tutti i moduli sono necessari: *Super Mario Galaxy*[6] ha bisogno solo del modulo di movimento, dato che i nemici devono semplicemente inseguire il giocatore. Altri giochi d'azione più complicati come *Dark Souls*[7] richiedono decisioni più complesse — attaccare, parare, muoversi o curarsi — ma non tengono conto delle azioni degli altri nemici, rendendo superflua la Strategia. Tutto ciò rientra nelle scelte di design: ogni videogioco vuole trasmettere emozioni diverse e offrire un diverso grado di sfida. AmbraIA opera soprattutto sul campo del Decision Making, cercando di non coprire un genere

preciso ma di creare un'IA che possa rendere gli NPC credibili in scene 3D differenti. Tradizionalmente, il Decision Making viene affrontato principalmente tramite due strutture: gli alberi decisionali e le macchine a stati finiti.

2.1.1 Alberi decisionali

Gli alberi decisionali sono strutture dati che vengono percorse ogni volta che deve essere presa una decisione. Sono formati da un insieme di nodi e da archi, cioè collegamenti direzionati tra due nodi che rappresentano le decisioni. Gli alberi sono caratterizzati dall'assenza di cicli. Le decisioni finali sono rappresentate dai nodi foglia, cioè nodi senza archi uscenti. Non appena si presenta la necessità di prendere una decisione, l'albero viene percorso a partire dal suo nodo iniziale, detto radice. Viene percorso un arco piuttosto che un altro a seconda degli input che l'agente riceve, come la percezione del mondo esterno o il suo stato interno, ad esempio fame o stanchezza. Una volta arrivati al nodo foglia, la decisione è stata presa e si esegue l'azione designata, come mostrato in Figura 2.2.

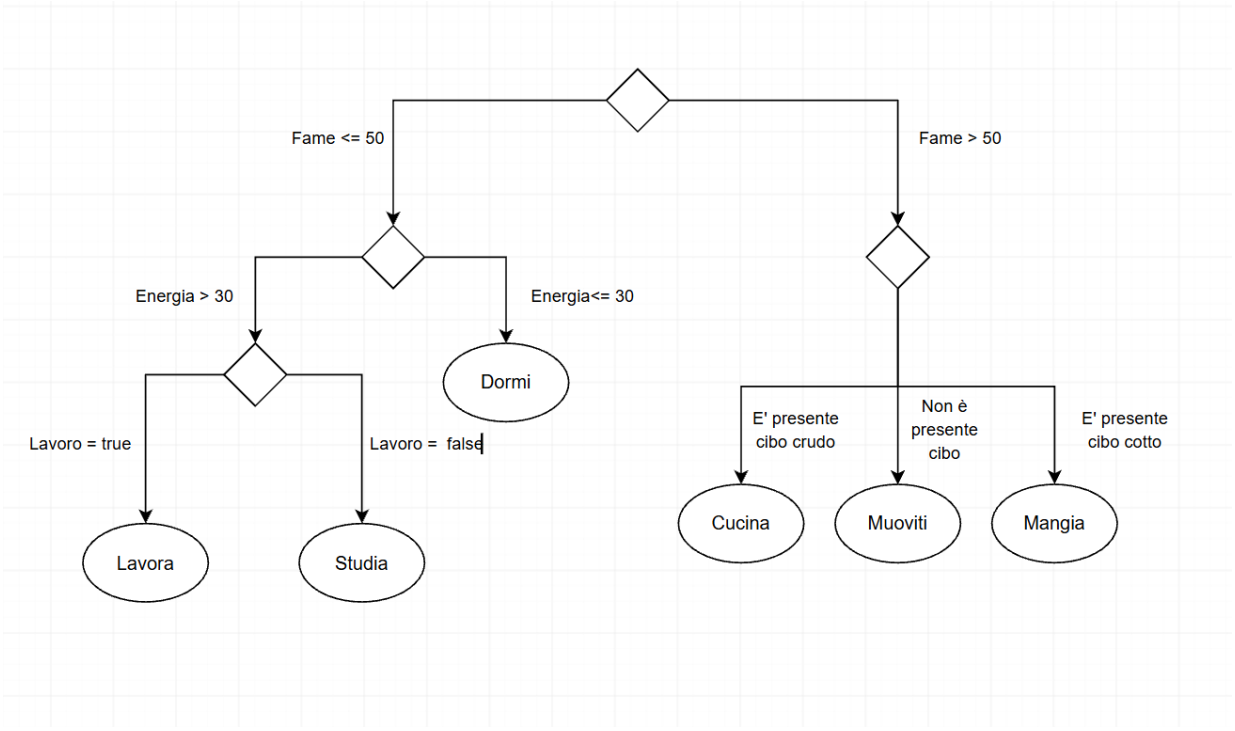


Figura 2.2: Esempio di albero decisionale per un NPC.

2.1.2 Macchine a stati finiti

Le macchine a stati finiti sono formalmente definite come una quintupla composta da:

- Un insieme di stati Q
- Un alfabeto Σ
- Uno stato iniziale q_0
- Un insieme di stati finali Q_f
- Una funzione di transizione $\delta : Q \times \Sigma \rightarrow Q$

Nel contesto videoludico gli stati rappresentano una situazione in cui si trova l'NPC (in guardia, affamato, impaurito) che lo porta ad agire in un certo modo. Lo stato iniziale è la situazione di partenza dell'NPC. L'alfabeto rappresenta una serie di input. La funzione di transizione controlla se una condizione si verifica in un certo stato e, se ciò avviene, cambia lo stato dell'NPC. Gli stati finali rappresentano situazioni in cui l'NPC ha raggiunto un obiettivo o ha completato un comportamento. Le decisioni vengono quindi prese in base allo stato in cui si trova l'NPC in un determinato momento, come mostrato in Figura 2.3.

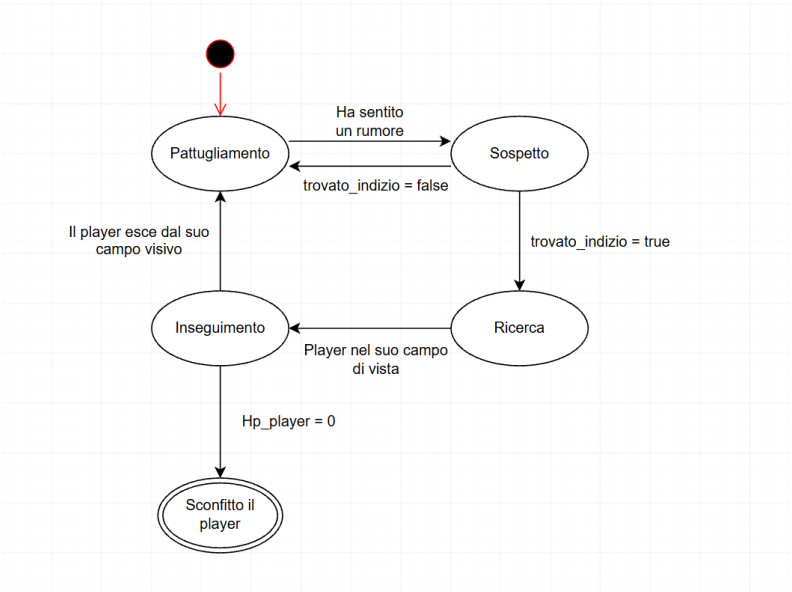


Figura 2.3: Esempio di macchina a stati finiti per un NPC.

2.1.3 Limiti degli approcci tradizionali

Questi approcci hanno molti vantaggi: sono strutturalmente semplici, possono essere trattati con valori primitivi e si possono combinare tra loro. Il limite più grande è la staticità di queste strutture: sono estremamente specifiche per il sistema in cui vengono implementate e non tengono conto delle azioni passate. Inoltre, ogni nuovo comportamento richiede la modifica manuale di stati, transizioni o rami dell'albero, rendendo difficile l'adattamento a contesti non previsti in fase di progettazione.

2.2 Gli LLM nei videogiochi: opportunità e sfide

L'intelligenza artificiale generativa e i Large Language Models rappresentano uno dei più grandi risultati dell'informatica moderna, trovando ormai applicazione nella quasi totalità degli ambiti tecnologici e lavorativi. Basati su architetture a reti neurali focalizzate sull'elaborazione e sulla generazione di linguaggio naturale, gli LLM si sono imposti non più come semplici strumenti di scrittura, ma come potenti motori di ragionamento, pianificazione e controllo.

2.2.1 LLM come strumenti per lo sviluppo di videogiochi

Anche nel mondo dei videogiochi gli LLM sono sbarcati, aiutando gli sviluppatori a creare strumenti che facilitano lo sviluppo. Un esempio è il progetto "LLMR: Real-time Prompting of Interactive Worlds using Large Language Models" di De La Torre et al. [8]. LLMR è un framework per la creazione di asset e scene 3D che utilizza un modello linguistico per la generazione di questi asset. Nell'architettura descritta nel paper è importante l'approccio human in the loop: l'utente inizia una conversazione con il modello per fornire tutte le informazioni necessarie alla generazione degli asset. A questo punto l'LLM genera i file C# che descrivono i vari elementi della scena. Una volta generati gli asset, l'utente può chiedere al modello modifiche automatiche oppure agire manualmente su di essi, prima di generare la scena completa.

2.2.2 LLM per il controllo degli NPC

Gli LLM inoltre assistono anche nella creazione di NPC. I comportamenti umani sono molto complessi, frutto di processi cognitivi articolati che comprendono l'analisi di ricordi ed esperienze passate, la percezione del mondo e lo stato fisiologico ed emotivo. I videogiochi che vogliono simulare i comportamenti umani, anche solo in parte, utilizzano per la maggior parte macchine a stati finiti enormi e molto specifiche per il gioco sviluppato; un esempio già citato è *Tomodachi Life* [3]. Gli LLM possono essere utilizzati per semplificare queste architetture.

Nel paper scientifico "LLM-Driven NPCs: Cross-Platform Dialogue System for Games and Social Platforms" di Song [9] viene proposto un prototipo di NPC che dialoga tramite LLM con il giocatore sia in-game tramite Unity, sia fuori dal gioco su Discord. Parlare nel gioco aumenta un parametro che indica il gradimento dell'NPC, cioè il grado di familiarità tra il giocatore e il personaggio. Più il gradimento è alto, più l'NPC sarà confidente verso il giocatore. In questo caso l'LLM ha il solo compito di generare output testuali in base agli input testuali del giocatore.

I Large Language Models però sono stati usati anche per elaborare decisioni più complesse, come in "Generative Agents: Interactive Simulacra of Human Behavior" di Park et al. [10], in cui viene progettata e implementata un'architettura che simula una cittadina di persone, le quali interagiscono tra loro, hanno la loro routine e la loro personalità. L'architettura cerca di simulare alcuni processi cognitivi della mente umana, come il deposito di ricordi in un database, l'estrazione di ricordi in base alla loro importanza e recenza, o la riflessione periodica sulle proprie azioni. Questo è un esempio di IA di strategia e decision making, poiché gli abitanti della cittadina interagiscono tra loro e prendono decisioni in base al mondo circostante e al loro stato attuale.

2.2.3 Sfide e limiti degli LLM

Nonostante le opportunità offerte, gli LLM presentano anche diverse criticità. Possono essere soggetti ad allucinazioni, generando informazioni non veritiere o incoerenti con il contesto; i tempi di esecuzione possono essere elevati, soprattutto quando il modello viene eseguito localmente; inoltre, in assenza di un'architettura

adeguata, tendono a ripetere le stesse risposte o gli stessi comportamenti. Queste sfide rendono necessario progettare sistemi che affianchino all'LLM componenti deterministiche in grado di gestire memoria, coerenza e affidabilità.

AmbraIA prende ispirazione dalle architetture sopraccitate e cerca di creare un NPC che possa interagire in un ambiente 3D di Unity, prendendo decisioni coerenti con il suo stato fisiologico e la sua percezione dell'ambiente. AmbraIA non è una struttura pensata per un videogioco o un ambiente specifico, grazie a un sistema automatico di matching tra *GameObject* e interazioni/animazioni definite dallo sviluppatore. L'LLM viene utilizzato da AmbraIA per pianificare le azioni, mappare le azioni in C# e generare riflessioni periodiche. Tutti questi aspetti vengono approfonditi nel Capitolo 3.

CAPITOLO 3

AmbraIA: dall'analisi del problema all'architettura

AmbraIA è un'architettura cognitiva per agenti autonomi in ambienti 3D, supportata da modelli linguistici generativi. Questa architettura è divisa in vari moduli, ognuno con il compito di preparare i dati per il modulo successivo; il risultato finale viene inviato a Unity, dove prende la forma di uno script C# che permette all'agente di muoversi e interagire con la scena. Prima di descrivere in dettaglio ogni modulo, partiamo dalla definizione del problema dell'NPC che interagisce con l'ambiente e dalla descrizione dell'ambiente stesso.

3.1 Specifica PEAS

Per descrivere formalmente l'agente e il suo ambiente, si utilizza il modello PEAS (Performance, Environment, Actuators, Sensors), uno standard nella progettazione di agenti intelligenti.

Performance Le misure di prestazione adottate per valutare l'operato dell'agente sono:

- Coerenza delle azioni con lo stato fisiologico
- Varietà dei comportamenti nel tempo

- Tasso di successo della compilazione del codice C# generato
- Tempo di risposta del server, cioè la latenza tra richiesta e piano generato
- Adattabilità a scene nuove, cioè il sistema configura scene non preconfigurate correttamente

Environment Gli elementi che formano l'ambiente sono:

- Una scena 3D Unity con oggetti interagibili (cibo, mobili, oggetti decorativi)
- Proprietà specifiche per ogni oggetto (nome, tag, azione disponibile, effetti fisiologici)

Actuators Gli attuatori disponibili dall'agente sono:

- Movimento verso un oggetto (`MoveToTargetAndWait`)
- Interazione con un oggetto (`InteractWithAndWait`)
- Modifica dello stato fisiologico come conseguenza delle interazioni

Sensors I sensori attraverso i quali l'agente riceve input percettivi sono:

- Lista degli oggetti nel raggio di percezione (nome, distanza, azione, effetto)
- Stato fisiologico corrente (fame, stanchezza)
- Cronologia delle azioni più importanti e recenti
- Memoria a lungo termine

3.2 Caratteristiche dell'ambiente

L'ambiente in cui opera AmbraIA presenta le seguenti caratteristiche:

Singolo agente Il problema è modellato come un problema a singolo agente. Non sono presenti altri NPC o agenti concorrenti durante l'esecuzione, sebbene l'architettura sia predisposta per future estensioni multi-agente.

Parzialmente osservabile L'agente percepisce solo gli oggetti entro un raggio di percezione configurabile, non l'intera scena. Inoltre, non ha accesso diretto allo stato interno degli oggetti.

Semi-deterministico Le interazioni con gli oggetti hanno effetti deterministici: mangiare una mela riduce sempre la fame di un valore fisso. Tuttavia, il processo decisionale basato su LLM introduce non-determinismo nella scelta delle azioni.

Sequenziale Il processo decisionale non è suddiviso in episodi indipendenti, ma consiste in una sequenza continua di pianificazione, esecuzione e aggiornamento della memoria. Le decisioni hanno conseguenze a lungo termine perché influenzano i ricordi che guideranno le scelte future.

Statico L'ambiente è statico durante l'esecuzione di un piano: gli oggetti non cambiano posizione o proprietà mentre l'NPC esegue le azioni.

Discreto Lo spazio degli stati è discreto. Le azioni disponibili sono in numero finito e lo stato fisiologico è rappresentato da valori normalizzati tra 0 e 1.

3.3 Analisi del problema

Il problema può essere formalizzato come un problema di pianificazione continua in un ambiente parzialmente osservabile, dove ogni stato è definito dalla combinazione di posizione dell'NPC, stato fisiologico, oggetti percepiti e contenuto del `Memory Stream`, cioè della memoria persistente.

Stati Configurazione corrente dell'NPC: posizione, fame, stanchezza e ricordi accumulati.

Stato iniziale NPC posizionato nella scena con fame e stanchezza iniziali e `Memory Stream` vuoto.

Azioni Movimento verso un oggetto oppure interazione con un oggetto.

Spazio degli stati Tutte le possibili combinazioni di posizioni, livelli fisiologici, oggetti percepiti e ricordi accumulati.

Test obiettivo Non esiste uno stato obiettivo finale. L'NPC deve mantenere un comportamento coerente e adattivo indefinitamente.

Il problema presenta le seguenti caratteristiche:

- Spazio degli stati potenzialmente illimitato, a causa della crescita continua del Memory Stream.
- Assenza di una soluzione ottima calcolabile: non esiste un comportamento "perfetto".
- Necessità di bilanciare bisogni multipli simultaneamente (fame e stanchezza).
- Necessità di mantenere coerenza temporale, senza contraddire i ricordi e le riflessioni.

Per questi motivi, il problema non può essere risolto tramite alberi decisionali predefiniti o macchine a stati finite tradizionali, ma risulta adatto all'uso di LLM combinati con un'architettura di memoria, retrieval e riflessione.

3.4 Architettura del modulo Unity

AmbraIA è suddivisa in due macro moduli: il modulo Unity e il modulo server. Il modulo Unity ha il compito di raccogliere i dati di input dall'ambiente e di gestire i dati di output, cioè le azioni che l'NPC deve eseguire. È composto da quattro sotto-moduli principali, implementati come script C#:

- `NPCController`: permette all'NPC di muoversi e interagire con gli oggetti della scena.
- `SceneAnalyzer`: analizza la scena e invia i dati al server.
- `InteractableObject`: permette ai *GameObject* di diventare interagibili.
- `RuntimeCompiler`: compila a runtime il codice C# generato dal server e lo attacca all'NPC.

La Figura 3.1 mostra una panoramica del lato Unity.

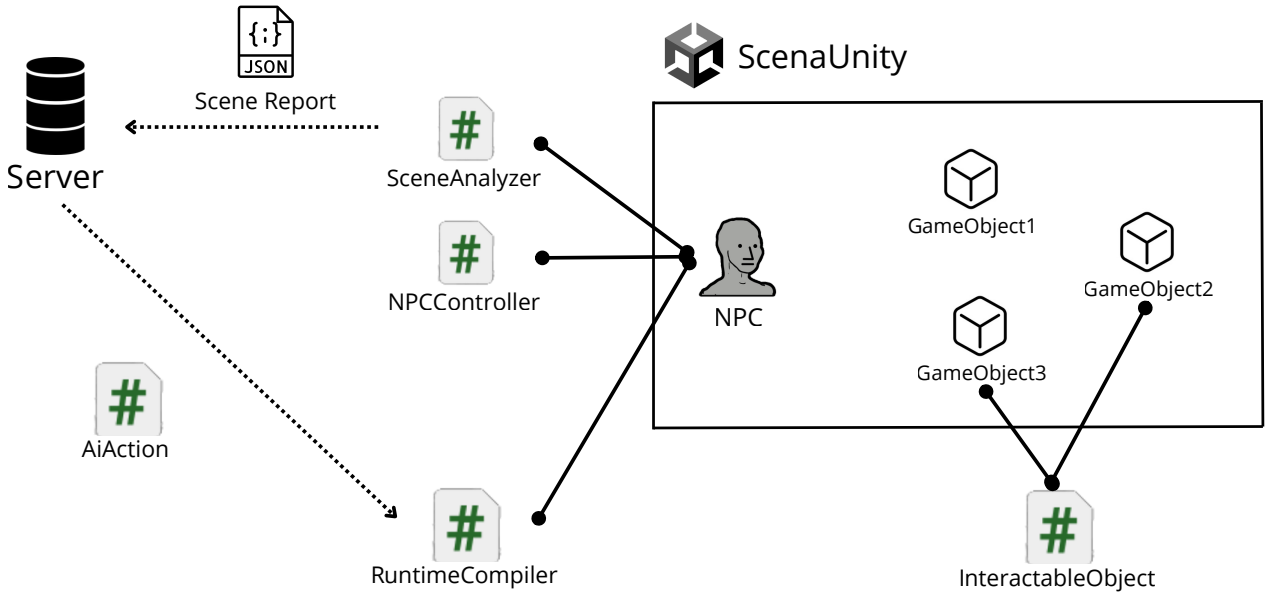


Figura 3.1: Panoramica del lato Unity di AmbraIA.

3.4.1 NPCController

Questo componente permette di eseguire il piano di azioni compilato dal `RuntimeCompiler`. Fornisce i metodi per far muovere l'NPC e per interagire con gli oggetti interagibili, attivando animazioni ed effetti. Il movimento viene gestito tramite il NavMesh, una rappresentazione semplificata della geometria della scena che indica le aree calpestabili e gli ostacoli. In questo modo l'agente può calcolare percorsi validi evitando muri e oggetti non attraversabili. Una volta arrivato nei pressi dell'oggetto con cui interagire, l'NPC attiva l'animazione e l'effetto specifico dell'interazione, descritti dall'`InteractableObject`.

L'`NPCController` contiene inoltre i dati dello stato interno dell'NPC, che in questo caso sono fame e stanchezza: questi rappresentano i bisogni prioritari che l'NPC deve soddisfare. Entrambi sono rappresentati da valori decimali compresi

tra 0 e 1, dove 0 indica uno stato ottimale e 1 un bisogno critico. Fame e stanchezza aumentano gradualmente nel tempo: la fame cresce costantemente, mentre la stanchezza aumenta più rapidamente quando l'NPC si muove e più lentamente quando è fermo. Le interazioni con gli oggetti possono ridurre o aumentare questi valori.

3.4.2 InteractableObject

`InteractableObject` è il modulo che rappresenta l'interazione e viene associato ai *GameObject* che possono essere interagiti. Contiene quattro informazioni principali:

- il nome dell'interazione (ad esempio "Usa" o "Mangia");
- l'animazione da attivare;
- la coordinata spaziale in cui l'NPC deve trovarsi per interagire con quell'oggetto;
- l'effetto, cioè il valore di modifica di fame e stanchezza dopo l'interazione.

Se a un *GameObject* non è associato un `InteractableObject`, questo non verrà considerato come oggetto interagibile. Il componente include inoltre un campo opzionale, `oggettoDaAttivare`, che permette di attivare un secondo *GameObject* al termine dell'interazione: questo consente di realizzare interazioni a più step, come nel caso del fornello che, una volta usato, fa apparire un piatto cotto.

3.4.3 SceneAnalyzer

`SceneAnalyzer` è il modulo che ha il compito di inviare i dati della scena al server tramite richiesta HTTP con endpoint `/npc/decide`. Il componente definisce il raggio di percezione dell'NPC: tutti gli oggetti interagibili che ricadono entro questo raggio vengono inseriti in una lista, dove ognuno è descritto da nome, tag, azione disponibile, effetto fisiologico e distanza dall'NPC. Nel calcolo dell'effetto, `SceneAnalyzer` legge direttamente i valori di `fameEffect` ed `energyEffect` dall'`InteractableObject` e li classifica come *hunger*, *energy*, *both* o *none*.

Oltre agli oggetti vicini, l'Analyzer invia al server anche lo stato fisiologico corrente dell'NPC, le ultime azioni eseguite e le riflessioni accumulate. Se l'NPC ha una fame o una stanchezza maggiore o uguale a 0.7, l'Analyzer notificherà la criticità. In questo modo il server ha tutte le informazioni necessarie per pianificare le azioni successive. Se il file C# generato dal server contiene un errore di compilazione, lo `SceneAnalyzer` notifica l'errore al server e richiede un nuovo tentativo: questo meccanismo di retry può avvenire fino a un massimo di tre volte, sia per errori di compilazione sia per errori HTTP.

Quando un'azione viene completata, lo `SceneAnalyzer` invia una notifica al server tramite l'endpoint `/npc/action-done`, in modo che l'azione venga registrata nella memoria a lungo termine. Questa comunicazione avviene separatamente dalla richiesta di pianificazione, così il `Memory Stream` rimane sempre aggiornato.

3.4.4 RuntimeCompiler

`RuntimeCompiler` ha il compito di compilare il piano d'azione, scritto in codice sorgente C# e generato dal server, per poi attaccarlo all'NPC. Unity normalmente compila tutto il codice a tempo di build, ma in questo caso il codice non esiste finché il server non lo genera, quindi è necessario compilarlo durante l'esecuzione del gioco. Per farlo, il componente utilizza Roslyn, il compilatore C# ufficiale di Microsoft.

Il processo avviene interamente in memoria: se la compilazione ha successo, la classe generata viene attaccata al *GameObject* dell'NPC, sostituendo quella del piano precedente. Se invece la compilazione fallisce, `RuntimeCompiler` restituisce una stringa con gli errori riscontrati, che `SceneAnalyzer` invierà al server per richiedere una correzione.

3.5 Architettura del modulo server

Il server ha il compito di elaborare i dati ricevuti da Unity e di generare un piano d'azione sotto forma di file C#, che viene poi rimandato a Unity. È stato implementato interamente in Python. Per orchestrare i diversi sotto-moduli del server ho scelto di utilizzare LangGraph, un framework open source sviluppato da

LangChain. LangGraph permette di costruire workflow complessi con agenti di AI generativa, gestendo in modo chiaro il flusso di dati tra un modulo e l'altro. Il server è composto da sei moduli principali:

- `Memory Stream`: memoria persistente che registra tutte le esperienze dell'NPC.
- `Retrieval`: recupera dal `Memory Stream` i ricordi più rilevanti per la situazione attuale.
- `Planner`: genera un piano d'azione gerarchico basato sulla scena, sui ricordi e sulle riflessioni, tramite una chiamata a un LLM.
- `Builder`: traduce il piano in codice C# eseguibile da Unity, sempre tramite LLM.
- `Inspector`: valida il codice generato, verificando che non contenga errori o placeholder.
- `Reflector`: analizza periodicamente le esperienze recenti e produce riflessioni di alto livello, tramite una chiamata all'LLM.

La Figura 3.2 mostra una panoramica del lato server.

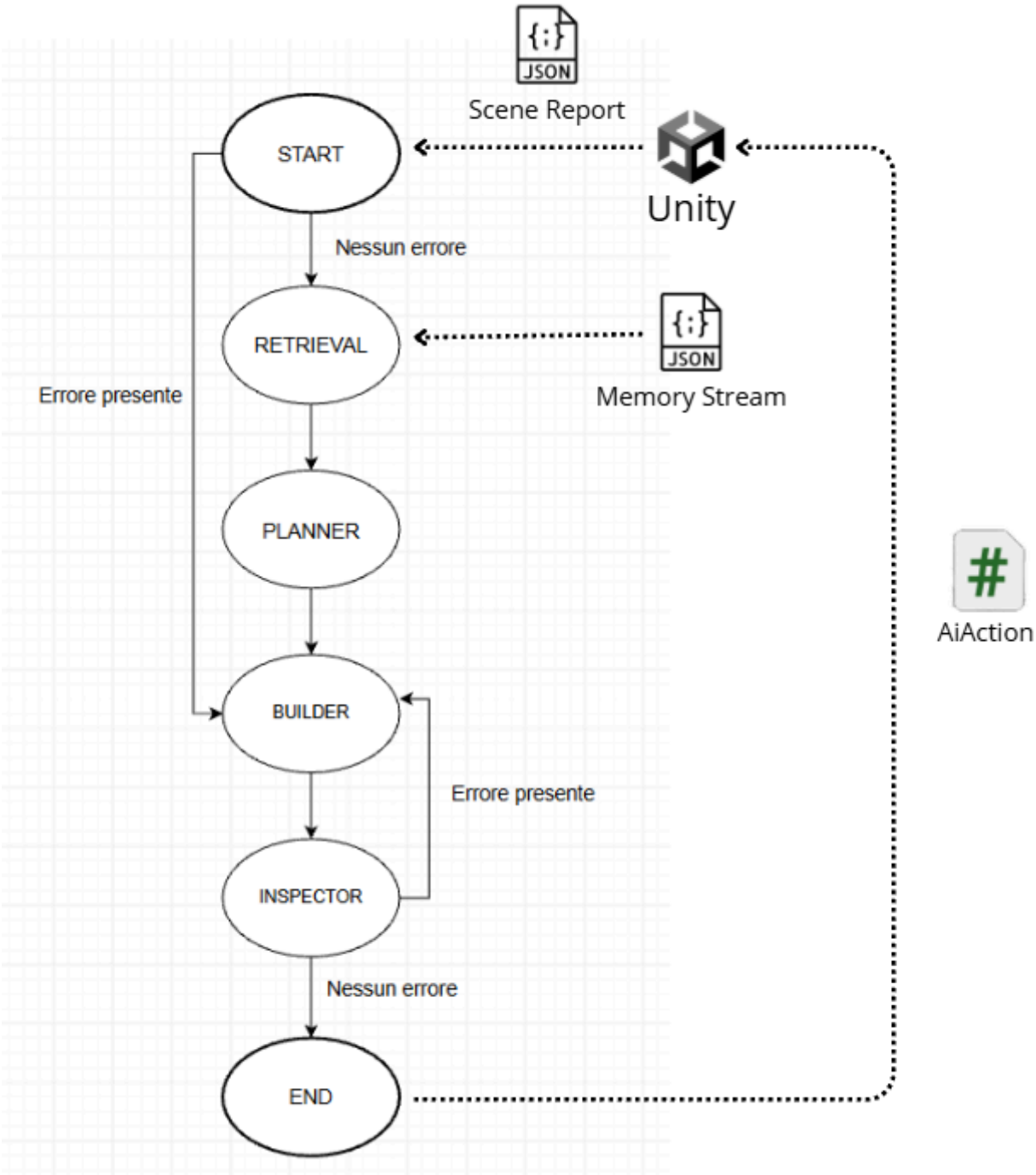


Figura 3.2: Panoramica del lato server di AmbraIA.

3.5.1 Memory Stream

Il `MemoryStream` rappresenta la memoria permanente dell'architettura. Consiste in un file JSON in cui sono raccolti tutti i "ricordi" dell'NPC, cioè le sue azioni passate. Questo serve a simulare i ricordi umani e aiuta a evitare comportamenti ripetitivi,

incoerenti, o che in passato hanno portato a effetti negativi. Ogni ricordo è composto da:

- `id`: identificativo alfanumerico del ricordo;
- `content`: descrizione testuale del ricordo;
- `timestamp`: momento in cui è avvenuto il ricordo;
- `importance`: numero intero da 1 a 10 che indica l'importanza del ricordo. Non tutte le esperienze hanno lo stesso valore: mangiare una mela quando si ha fame è più significativo che mangiarla quando si è già sazi. Per questo il punteggio viene assegnato dall'LLM al momento della creazione del ricordo, valutando l'impatto sullo stato fisico, la novità dell'evento e l'utilità per le decisioni future. Questo valore influenza il `Retrieval`, dando più peso ai ricordi ritenuti più importanti;
- `type`: tipo del ricordo, che può essere `action` o `reflection`;
- `embedding`: vettore di 768 numeri che rappresenta il significato del testo. Viene generato tramite `nomic-embed-text`, un modello eseguito localmente che trasforma una frase in una rappresentazione numerica. Due frasi con significato simile avranno vettori vicini nello spazio, anche se non condividono le stesse parole: ad esempio "Ho mangiato una mela" e "Ho consumato del cibo" producono embedding molto simili. Questo permette al `Retrieval` di confrontare i ricordi con la situazione attuale in modo semantico, senza dover cercare parole chiave.

La Lista 3.1 mostra un esempio di ricordo salvato nel `MemoryStream`.

Listing 3.1: Esempio di ricordo nel Memory Stream.

```
1        {
2                "id": "mem_a3f2c1d4",
3                "content": "Mangia su Mela",
4                "timestamp": 1726000000.0,
5                "importance": 6,
```

```
6                   "type": "action",
7                   "embedding": [0.12, -0.34, 0.78, "...altri 765
                        valori..."]
8           }
```

3.5.2 Retrieval

Il `Retrieval` è il modulo che si occupa di recuperare dal `MemoryStream` i ricordi più utili per la situazione attuale. L'LLM, da solo, non può ricevere l'intera memoria dell'NPC: dopo diverse ore di gioco il numero di ricordi accumulati diventa troppo elevato per essere inserito in un singolo prompt. È quindi necessario un meccanismo di selezione che scelga solo i ricordi più rilevanti; d'altronde anche la mente umana estrae specifici ricordi in specifiche situazioni.

Il `Retrieval` estrae i primi dieci ricordi in base a un punteggio calcolato combinando tre fattori:

- **Recency**: quanto è recente il ricordo. Un'esperienza appena vissuta è spesso più utile e adatta al contesto attuale di una di diverse ore fa. Il decadimento segue una funzione iperbolica, così i ricordi invecchiano rapidamente all'inizio e più lentamente col passare del tempo;
- **Importance**: quanto è importante il ricordo, secondo il punteggio da 1 a 10 assegnato dall'LLM al momento della creazione del ricordo;
- **Relevance**: quanto il ricordo è semanticamente simile alla situazione attuale. Per calcolarla, ogni ricordo e la situazione corrente vengono trasformati in embedding, cioè vettori di 768 numeri che ne rappresentano il significato. Due testi con significato simile producono vettori vicini nello spazio, anche se non condividono le stesse parole. La similarità tra i due vettori si misura tramite la similarità del coseno, che restituisce un valore tra 0 e 1: valori vicini a 1 indicano che il ricordo è semanticamente pertinente alla situazione attuale, valori vicini a 0 indicano che è poco rilevante.

Il punteggio finale è una combinazione pesata dei tre fattori, secondo la formula:

$$\text{score} = \alpha_{\text{recency}} \cdot \text{recency} + \alpha_{\text{importance}} \cdot \text{importance} + \alpha_{\text{relevance}} \cdot \text{relevance} \quad (3.5.1)$$

dove i pesi sono $\alpha_{\text{recency}} = 0.3$, $\alpha_{\text{importance}} = 0.3$ e $\alpha_{\text{relevance}} = 0.4$. Il peso maggiore dato alla relevance riflette l'importanza della pertinenza contestuale: se l'NPC ha fame, un ricordo legato al cibo è più utile di un ricordo recente ma su un argomento diverso.

La formula completa per il calcolo del punteggio, tenendo conto di tutti e tre i fattori, è la seguente:

$$\text{score}(m) = \alpha_r \cdot \frac{1}{1 + \lambda \cdot \Delta t_m} + \alpha_i \cdot \frac{i_m}{10} + \alpha_v \cdot \cos(\vec{e}_m, \vec{e}_q) \quad (3.5.2)$$

dove:

- Δt_m è il tempo trascorso (in ore) dal momento in cui il ricordo m è stato creato;
- $\lambda = 0.99$ è il fattore di decadimento;
- i_m è l'importance del ricordo m;
- $\vec{e}_m$ è l'embedding del ricordo m;
- $\vec{e}_q$ è l'embedding della query, cioè la descrizione della situazione attuale.

I dieci ricordi con il punteggio più alto vengono passati al `Planner` come contesto storico, così che il piano generato tenga conto delle esperienze passate dell'NPC.

3.5.3 Planner

Il `Planner` è il primo nodo del grafo a chiamare l'LLM. Il suo compito è generare un piano d'azione in linguaggio naturale, che verrà poi utilizzato come input dal nodo successivo, il `Builder`. Riceve come input i ricordi estratti dal `Retrieval`, tutti gli oggetti interagibili presenti nella scena, lo stato fisiologico corrente dell'NPC, le azioni recenti e le riflessioni accumulate.

Il formato della risposta è un piano d'azione composto da un obiettivo principale e da un massimo di tre sotto-obiettivi, ciascuno con una priorità (alta, media o bassa).

Ogni sotto-obiettivo descrive l'azione da compiere e l'oggetto su cui agire, in modo che il `Builder` comprenda in che ordine devono essere eseguite le azioni.

La Lista 3.2 mostra un esempio di piano generato dal `Planner`.

Listing 3.2: Esempio di piano generato dal Planner.

```
1        GOAL: Mantenermi in salute
2
3        SOTTO-OBIETTIVO 1 [PRIORITA: ALTA]: Soddisfare la fame
4        - Azione: Vai a Mela
5        - Azione: Mangia su Mela
6
7        SOTTO-OBIETTIVO 2 [PRIORITA: MEDIA]: Riposarmi
8        - Azione: Vai a Sedia
9        - Azione: Siediti su Sedia
```

Il `Planner` tiene conto di alcune regole fondamentali: se la fame o la stanchezza superano la soglia critica di 0.7, il bisogno corrispondente diventa la priorità assoluta; le azioni già eseguite di recente non tendono ad essere ripetute; se le riflessioni suggeriscono di variare il comportamento, il piano deve riflettere questo cambiamento; se non ci sono bisogni urgenti, l'NPC deve esplorare nuovi oggetti o attività.

3.5.4 Builder

Il `Builder` converte il piano d'azione in linguaggio naturale, generato dal `Planner`, in codice C# eseguibile su Unity. Come il `Planner`, riceve in input la lista degli oggetti interagibili con le rispettive azioni valide e le distanze dall'NPC. In base alle azioni decise dal `Planner`, il `Builder` mappa ogni interazione con un massimo di due istruzioni:

- `MoveToTargetAndWait`: muove l'NPC verso l'oggetto target, se questo si trova a una distanza superiore a un'unità;
- `InteractWithAndWait`: obbligatoria per far avvenire l'interazione, avvia l'animazione e applica l'effetto fisiologico corrispondente.

La Lista 3.3 mostra un esempio di codice C# generato dal `Builder`.

Listing 3.3: Esempio di codice C# generato dal Builder.

```
using UnityEngine;
using System.Collections;

public class AiAction : MonoBehaviour
{
        void Start()
        {
                StartCoroutine(ExecutePlan());
        }

        IEnumerator ExecutePlan()
        {
                NPCController controller = GetComponent<
                   NPCController>();
                if (controller == null) yield break;

                // SOTTO-OBIETTIVO 1: Soddisfare la fame [
                   PRIORITA: ALTA]
                yield return StartCoroutine(controller.
                   MoveToTargetAndWait("Mela"));
                yield return StartCoroutine(controller.
                   InteractWithAndWait("Mela", "Mangia"));

                // SOTTO-OBIETTIVO 2: Riposarmi [PRIORITA:
                   MEDIA]
                yield return StartCoroutine(controller.
                   MoveToTargetAndWait("Sedia"));
                yield return StartCoroutine(controller.
                   InteractWithAndWait("Sedia", "Siediti"))
                   ;

                Destroy(this);
```

```
25                          yield break;
26                  }
27          }
```

Il `Builder` opera in modalità deterministica, con temperatura impostata a zero, per ridurre al minimo la variabilità dell'output. Il codice generato è vincolato a una struttura fissa: la classe deve chiamarsi `AiAction`, deve ereditare da `MonoBehaviour` e deve usare esclusivamente i due metodi esposti dall'`NPCController`. Al termine dell'esecuzione, la classe si autodistrugge tramite `Destroy(this)`, così che il componente non si accumuli sull'NPC tra un piano e il successivo.

3.5.5 Inspector

L'`Inspector` è l'ultimo nodo del grafo e ha il compito di controllare che il codice generato dal `Builder` non contenga errori gravi. Se il controllo fallisce, il codice viene rimandato al `Builder` per un ulteriore tentativo, fino a un massimo di tre.

L'`Inspector` verifica tre categorie di problemi:

- **Placeholder**: controlla che il codice non contenga pseudo-codice, commenti al posto di istruzioni reali (es. `if (/* condizione */)`), oppure testi come "TODO" o "da implementare";
- **Nomi degli oggetti**: estrae i nomi degli oggetti usati nelle chiamate a `MoveToTargetAndW` e `InteractWithAndWait` e verifica che corrispondano a oggetti realmente presenti nella scena;
- **Nomi dei componenti**: controlla che il nome del controller utilizzato nel codice sia corretto, cioè che sia `NPCController` e non una variante inventata dall'LLM.

Può capitare che il codice contenga altri tipi di errore che l'`Inspector` non è in grado di rilevare, come errori di sintassi o di tipo. In questo caso, quando Unity tenta di compilare il codice, genera un errore e lo rispedisce al server. Il grafo riparte direttamente dal `Builder`, saltando `Retrieval` e `Planner`, ricevendo l'errore nel prompt per correggere il codice. In questo modo, l'`Inspector` ha il compito

principale di individuare gli errori immediati e di velocizzare il flusso di lavoro, evitando di far arrivare a Unity codice che sarebbe sicuramente scartato.

3.5.6 Reflector

Il `Reflector` non è un nodo del grafo, ma un modulo indipendente che viene eseguito ogni cinque azioni completate. Come input riceve i ricordi più recenti del `Memory Stream`, in particolare le ultime venti azioni e osservazioni. Il `Reflector` chiede all'LLM di generare una o più riflessioni di alto livello sui comportamenti recenti dell'NPC, identificando pattern, abitudini ricorrenti o comportamenti poco coerenti.

Le riflessioni generate vengono salvate nel `Memory Stream` come ricordi di tipo `reflection`, con un punteggio di importanza fissato a 9. In questo modo, grazie alla formula del `Retrieval`, tenderanno a essere recuperate spesso e a influenzare le pianificazioni successive; di conseguenza il `Planner` terrà conto di queste osservazioni.

La Lista 3.4 mostra un esempio di riflessione generata dal `Reflector`.

Listing 3.4: Esempio di riflessione generata dal Reflector.

```
1  L'NPC tende a mangiare sempre la stessa mela.
2  Dovrebbe esplorare altre fonti di cibo disponibili nella
       scena.
```

3.6 Configurazione scena

AmbraIA prevede una configurazione della scena automatica. L'utente deve inserire la proprietà *static* sul pavimento e su tutti i *GameObject* che sono considerati ostacoli per il movimento. Deve poi creare dei Template per gli oggetti interagibili, creabili dal menù a tendina degli asset, in cui specifica nome, animazione ed effetto, più due campi opzionali dove può aggiungere delle parole chiave per il nome o per il tag degli oggetti. Ad esempio, se l'azione è "siediti" può inserire parole chiave come "sedia, divano, letto, poltrona".

A questo punto, dalla barra in alto, sotto la voce "NPC" può avviare l'analisi che mappa tutti gli oggetti con un tag come interagibili e associa a ciascuno un `InteractableObject` in base al template. L'associazione può avvenire in due modi: per corrispondenza di parole chiave, oppure, in mancanza di corrispondenza, tramite una chiamata all'LLM che, ricevendo la lista di oggetti e i template disponibili, deciderà a quale template associare ogni oggetto.

Perché l'NPC venga riconosciuto dal sistema, il suo *GameObject* deve avere un componente `NPCController` attaccato, oltre a `SceneAnalyzer`, `RuntimeCompiler`, `NavMeshAgent` e `Animator`. Il tool cerca un `NPCController` nella scena per verificare se l'NPC è già presente e, se non lo trova, lo istanzia automaticamente dalla posizione $(0, 0, 0)$.

A questo punto la scena è pronta, ma nulla vieta all'utente di apportare ulteriori modifiche alle interazioni o alle associazioni qualora non fosse soddisfatto del risultato.

CAPITOLO 4

Sperimentazione e Risultati

4.1 Setup sperimentale

L'architettura è stata testata principalmente su una singola scena che funge da soggiorno, contenente sei oggetti interagibili: un PC, una sedia, un divano, una console, un bicchiere di vino e una ciotola di cibo. La Tabella 4.1 riassume i dettagli della configurazione utilizzata durante i test.

Tabella 4.1: Configurazione del setup sperimentale.

Parametro	**Valore**
Ambiente	
Tipo di scena	Soggiorno
Oggetti interagibili	6 (PC, sedia, divano, console, bicchiere di vino, ciotola di cibo)
Template	
Template creati	PC, Cibo, Videogioco, Bevande, Letto, Sedia, Libro
Effetti delle interazioni	
PC (Lavora)	fame: 0, energia: +0.3
Sedia (Siediti)	fame: 0, energia: -0.3
Divano (Siediti)	fame: 0, energia: -0.3
Console (Gioca)	fame: 0, energia: 0
Bicchiere di vino (Bevi)	fame: -0.15, energia: 0
Ciotola di cibo (Mangia)	fame: -0.55, energia: 0
Modelli	
Modello LLM	llama3.1
Modello di embedding	nomic-embed-text
Dimensione embedding	768
Parametri fisiologici	
Consumo fame passivo	0.0007/s
Consumo energia movimento	0.007/s
Consumo energia da fermo	0.0004/s
Soglia critica	0.7
Ricordi recuperati (top-N)	10
Pesi Retrieval	0.3 recency, 0.3 importance, 0.4 relevance

La Figura 4.1 mostra la scena utilizzata durante i test, con l'NPC al centro della scena.

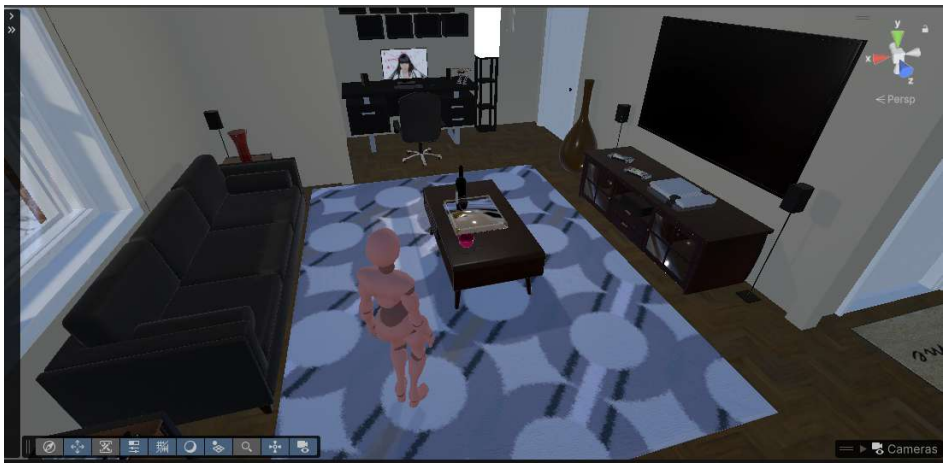


Figura 4.1: La scena di test con l'NPC.

4.2 Criteri di valutazione

Per valutare i risultati dell'architettura bisogna considerare principalmente metriche qualitative rispetto a quantitative, poiché il comportamento di un NPC non è misurabile con metriche tradizionali come accuracy o precision. La credibilità e la coerenza di un agente emergono dall'osservazione del comportamento nel tempo, non dal calcolo di un singolo valore numerico. Fanno eccezione i tempi di risposta e il tasso di successo della compilazione, che sono intrinsecamente quantitativi. Le metriche utilizzate per valutare il sistema sono riassunte nella Tabella 4.2.

Tabella 4.2: Criteri di valutazione adottati.

Criterio	Descrizione	Tipo
Coerenza delle azioni	L'NPC si definisce coerente se compie azioni che riflettono il suo stato fisiologico, ad esempio mangia se ha fame, oppure si intrattiene o studia se non ha bisogno di mangiare o riposare.	Qualitativa
Varietà comportamentale	L'NPC ha una buona varietà comportamentale se non tende a ripetere le stesse azioni in presenza di oggetti diversi ma con effetto simile.	Qualitativa
Tasso di successo compilazione	Rapporto tra il numero di compilazioni riuscite e il numero di compilazioni totali.	Quantitativa
Latenza	Differenza di tempo tra la richiesta e la risposta.	Quantitativa
Adattabilità	Il sistema si adatta correttamente a nuove scene.	Qualitativa

4.3 Analisi dei risultati

4.3.1 Coerenza delle azioni

L'NPC ha sempre dato priorità ai propri bisogni critici. Quando la fame o la stanchezza superavano la soglia critica, il `Planner` assegnava loro priorità alta nel piano, privilegiando le azioni con effetti maggiori sul bisogno in questione. Questo comportamento ha ridotto la varietà delle azioni nei momenti di criticità, ma ha confermato il funzionamento della memoria: i ricordi recuperati dal `Retrieval`

hanno permesso all'NPC di scegliere le interazioni che in passato avevano prodotto i risultati migliori.

4.3.2 Varietà comportamentale

Il comportamento dell'NPC è rimasto vario nel complesso. Quando lo stato fisiologico non era critico, l'NPC ha esplorato nuove interazioni, provando oggetti diversi presenti nella scena. Si è comunque osservata una tendenza a ripetere alcune azioni, in particolare quelle che migliorano lo stato fisiologico: questo comportamento è coerente con il funzionamento del sistema, perché il `Retrieval` tende a recuperare ricordi legati a esperienze positive, portando il `Planner` a preferire azioni già note come efficaci.

4.3.3 Tasso di successo della compilazione

Su circa trenta esecuzioni, la compilazione del codice C# generato dal Builder è fallita due volte. In entrambi i casi, il sistema ha gestito l'errore tramite il meccanismo di retry: il codice non compilato è stato rimandato al Builder con l'errore nel prompt, e il secondo tentativo ha prodotto codice valido. Il tasso di successo complessivo si attesta quindi intorno al 93%.

4.3.4 Adattabilità a scene nuove

L'adattabilità del sistema è stata valutata osservando il comportamento del meccanismo di configurazione automatica. Nella scena principale erano presenti sei oggetti interagibili: tre di questi sono stati associati ai template tramite matching automatico per parole chiave, mentre i restanti tre, non corrispondendo a nessuna parola chiave definita, sono stati inviati al server e configurati tramite una chiamata all'LLM. Il sistema di configurazione è stato testato anche su altre due scene, con esito positivo. Ogni configurazione è stata completata in circa 1440 ms in media. Questo dimostra che il sistema è in grado di associare template a oggetti non preconfigurati senza intervento manuale.

4.3.5 Latenza

La latenza misura il tempo che intercorre tra l'invio della richiesta da Unity e la ricezione del piano generato dal server. Nei test condotti, la latenza media per la generazione di un piano si è attestata intorno ai 57 secondi. Questo valore è dominato dalle due chiamate all'LLM, una per il `Planner` e una per il `Builder`, che insieme occupano la maggior parte del tempo totale di risposta.

La Tabella 4.3 mostra la latenza media per ogni nodo del grafo.

Tabella 4.3: Latenza media per nodo del grafo.

Nodo	**Tempo medio (s)**
Retrieval	[0.81]
Planner	[32.14]
Builder	[23.55]
Inspector	[0.02]
Totale	**56.52**

Un valore di latenza di questa entità non è compatibile con un utilizzo interattivo in tempo reale, ma risulta accettabile per una dimostrazione prototipale: la generazione del piano avviene in background mentre l'NPC completa il piano precedente, quindi il gioco non si blocca in attesa della risposta. Il valore dipende in larga misura dal modello linguistico utilizzato e dall'hardware su cui viene eseguito; modelli più piccoli o GPU dedicate potrebbero ridurre sensibilmente il tempo di risposta.

4.3.6 Considerazioni generali

I risultati osservati sono coerenti con gli obiettivi stabiliti. Il `Memory Stream`, il `Retrieval` e il `Reflector` hanno prodotto un comportamento vario e coerente con la situazione, e il meccanismo di retry ha garantito la robustezza del flusso di generazione del codice.

Nel complesso, i criteri di valutazione definiti nella Sezione 4.2 sono stati soddisfatti: l'NPC ha mostrato comportamenti coerenti con il proprio stato fisiologico, una

buona varietà comportamentale, un tasso di successo della compilazione pari al 93%, e la capacità di adattarsi a oggetti non preconfigurati. La latenza rimane il principale limite del prototipo, ma non compromette la validità dell'architettura e lascia spazio a ottimizzazioni future.

CAPITOLO 5

Conclusioni

5.1 Considerazioni finali

AmbraIA si configura come un'architettura cognitiva flessibile e indipendente da servizi cloud, pensata per integrarsi in scene 3D su Unity senza richiedere configurazioni complesse. I risultati ottenuti confermano il raggiungimento degli obiettivi prefissati: l'NPC è in grado di generare riflessioni e azioni variabili e coerenti con il proprio stato interno, offrendo un comportamento dinamico e personalizzabile. Pur emergendo alcune criticità legate alla latenza causata principalmente dalle chiamate agli LLM, il valore principale del lavoro non risiede nel singolo comportamento generato, ma nella struttura modulare e riproducibile che lo rende possibile. L'architettura offre così un solido punto di partenza per la prototipazione di NPC credibili, consentendo all'utente di modularne parametri fisiologici e interazioni senza la necessità di intervenire sul codice sorgente.

5.2 Sviluppi futuri

Il sistema è predisposto per implementare il modulo di Strategia, cioè una struttura che permetta la gestione multi-agente. In questo scenario, gli LLM potrebbero

essere utilizzati anche per permettere all'NPC di dialogare con l'utente o con altri NPC, arricchendo ulteriormente il comportamento.

Un altro lavoro futuro riguarda l'ottimizzazione dei tempi di risposta e quindi la riduzione della latenza, possibile magari utilizzando LLM più efficienti o modelli più piccoli.

Rendere l'ambiente da statico a dinamico potrebbe essere un ulteriore miglioramento, cioè fare in modo che gli oggetti con cui interagisce l'NPC cambino nel tempo. Nonostante non sia l'obiettivo principale di AmbraIA, la compatibilità con ambienti dinamici renderebbe il comportamento dell'NPC più realistico e adattabile a un numero maggiore di scene.

Un'ulteriore direzione di sviluppo riguarda l'estensione di AmbraIA ad altri motori grafici oltre a Unity, come Unreal Engine o Godot. L'architettura del server rimarrebbe invariata, mentre cambierebbe il modulo client che si occupa della comunicazione con l'ambiente 3D e dell'esecuzione delle azioni. Questo permetterebbe di portare l'architettura su una gamma più ampia di progetti.

Nel complesso, AmbraIA rappresenta un punto di partenza concreto per lo sviluppo di NPC credibili e adattivi, e le direzioni future delineate mostrano come l'architettura possa evolvere verso scenari sempre più complessi.

Il codice sorgente è disponibile su `https://github.com/girgio/AmbraIA`.

Bibliografia

[1] I. Millington and J. Funge, *Artificial Intelligence for Games*, 3rd ed. Boca Raton, FL: CRC Press, 2019. (Citato alle pagine iii, 4 e 5)

[2] Maxis, "The sims," 2000, videogioco. (Citato alle pagine 1 e 4)

[3] Nintendo, "Tomodachi life," 2013, videogioco. (Citato alle pagine 1 e 9)

[4] Atari, "Pong," 1972, videogioco. (Citato a pagina 4)

[5] Namco, "Pac-man," 1980, videogioco. (Citato a pagina 4)

[6] Nintendo, "Super mario galaxy," 2007, videogioco. (Citato a pagina 5)

[7] FromSoftware, "Dark souls," 2011, videogioco. (Citato a pagina 5)

[8] F. De La Torre, C. M. Fang, H. Huang, A. Banburski-Fahey, J. Amores Fernandez, and J. Lanier, "Llmr: Real-time prompting of interactive worlds using large language models," in *Proceedings of the CHI Conference on Human Factors in Computing Systems*, Honolulu, HI, USA, 2024. (Citato a pagina 8)

[9] L. Song, "Llm-driven npcs: Cross-platform dialogue system for games and social platforms," *arXiv preprint arXiv:2504.13928*, 2025. (Citato a pagina 9)

[10] J. S. Park, J. C. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, "Generative agents: Interactive simulacra of human behavior," in *Proceedings of*

the 36th Annual ACM Symposium on User Interface Software and Technology (UIST), 2023. (Citato a pagina 9)

Ringraziamenti

Arrivato a questo traguardo, desidero esprimere la mia gratitudine a tutte le persone che mi hanno accompagnato, sostenuto e incoraggiato in questi anni.

Hanno la mia gratitudine il Professor Fabio Palomba, il Dottor Antonio Della Porta e la Dottoressa Viviana Pentangelo per avermi assistito durante il periodo di stesura e per la loro preziosa disponibilità.

Ringrazio i miei genitori, De Caro Salvatore e Provvisiero Giovanna, per il supporto e la fiducia che mi hanno sempre dato. Senza di voi questo obiettivo sarebbe stato irraggiungibile.

Un ringraziamento a mia sorella Giulia De Caro, di cui sono orgoglioso e fiero, e alla mia sorellina Marta De Caro, la cui sola presenza a casa illumina le mie giornate.

Un pensiero di gratitudine va anche a mio zio Angelo Provvisiero, colui a cui in parte mi ispiro e che mi ha dato un grande aiuto durante i miei studi.

Un ringraziamento speciale va ai miei amici Marco Patellaro, Donato Nappi e Angelo Limonciello, i quali mi fanno sorridere ogni giorno e continuano ad alimentare la mia passione.

Infine, un riconoscimento a tutti coloro che hanno reso possibile questo risultato.

Grazie a tutti voi.